Tarefa 16: Produto Matriz-Vetor MPI

Escondendo Latência com Sobreposição de Computação

Werbert Arles de Souza Barradas

DCA3703 - Programação Paralela
UFRN - Universidade Federal do Rio Grande do Norte

Setembro de 2025

Introdução: O Problema

O objetivo deste trabalho é analisar a performance de uma implementação paralela para o cálculo do produto matriz-vetor, utilizando a biblioteca MPI (Message Passing Interface).

A operação matemática central é:

$$y = A \cdot x$$

Onde:

- A é uma matriz de dimensões $M \times N$.
- x é um vetor de tamanho N.
- y é o vetor resultante de tamanho M.

Objetivo: Avaliar como o tempo de execução varia com o aumento do tamanho da matriz e do número de processos.

Estratégia de Paralelização com MPI

A solução foi implementada seguindo uma estratégia de paralelismo de dados:

1. **Distribuição da Matriz (Scatter):**
   - A matriz A é dividida em blocos de linhas.
   - O processo 0 distribui um bloco de linhas para cada processo (incluindo ele mesmo) usando a função `MPI_Scatter`.
2. **Distribuição do Vetor (Broadcast):**
   - O vetor x completo é necessário por todos os processos.
   - O processo 0 envia uma cópia de x para todos os outros processos com a função `MPI_Bcast`.
3. **Cálculo Local e Agrupamento (Gather):**
   - Cada processo calcula uma parte do vetor de resultado y.
   - Os resultados parciais são enviados de volta para o processo 0, que os monta na ordem correta usando `MPI_Gather`.

Configuração do Benchmark

O experimento foi conduzido para avaliar a escalabilidade da solução.

Parâmetros Variados:

- **Tamanho da Matriz (M=N):** 1024 2048 4096 6144 8192 16384.
- **Número de Processos (P):** 2, 4, 8, 16, 32 e 64.

Métricas Coletadas:

- **Tempo de Execução (s):** Tempo total gasto (cálculo + comunicação).

Resultados do Benchmark (Parte 1 de 3)

Tamanho(M)	Tamanho(N)	Processos(P)	Tempo(s)
1024	1024	2	0.005144
1024	1024	4	0.006223
1024	1024	8	0.005868
1024	1024	16	0.003961
1024	1024	32	0.004128
1024	1024	64	0.005574
2048	2048	2	0.015974
2048	2048	4	0.013128
2048	2048	8	0.022243
2048	2048	16	0.008791
2048	2048	32	0.010243
2048	2048	64	0.012296

Resultados do Benchmark (Parte 2 de 3)

Tamanho(M)	Tamanho(N)	Processos(P)	Tempo(s)
4096	4096	2	0.056200
4096	4096	4	0.040949
4096	4096	8	0.045002
4096	4096	16	0.026021
4096	4096	32	0.024467
4096	4096	64	0.020109
6144	6144	2	0.122887
6144	6144	4	0.084915
6144	6144	8	0.078473
6144	6144	16	0.048340
6144	6144	32	0.046975
6144	6144	64	0.044552

Resultados do Benchmark (Parte 3 de 3)

Tamanho(M)	Tamanho(N)	Processos(P)	Tempo(s)
8192	8192	2	0.214889
8192	8192	4	0.150185
8192	8192	8	0.134295
8192	8192	16	0.081475
8192	8192	32	0.062831
8192	8192	64	0.069484
16384	16384	2	0.845109
16384	16384	4	0.571909
16384	16384	8	0.523586
16384	16384	16	0.197200
16384	16384	32	0.169794
16384	16384	64	0.153451

Resultados: Gráfico de Tempo de Execução

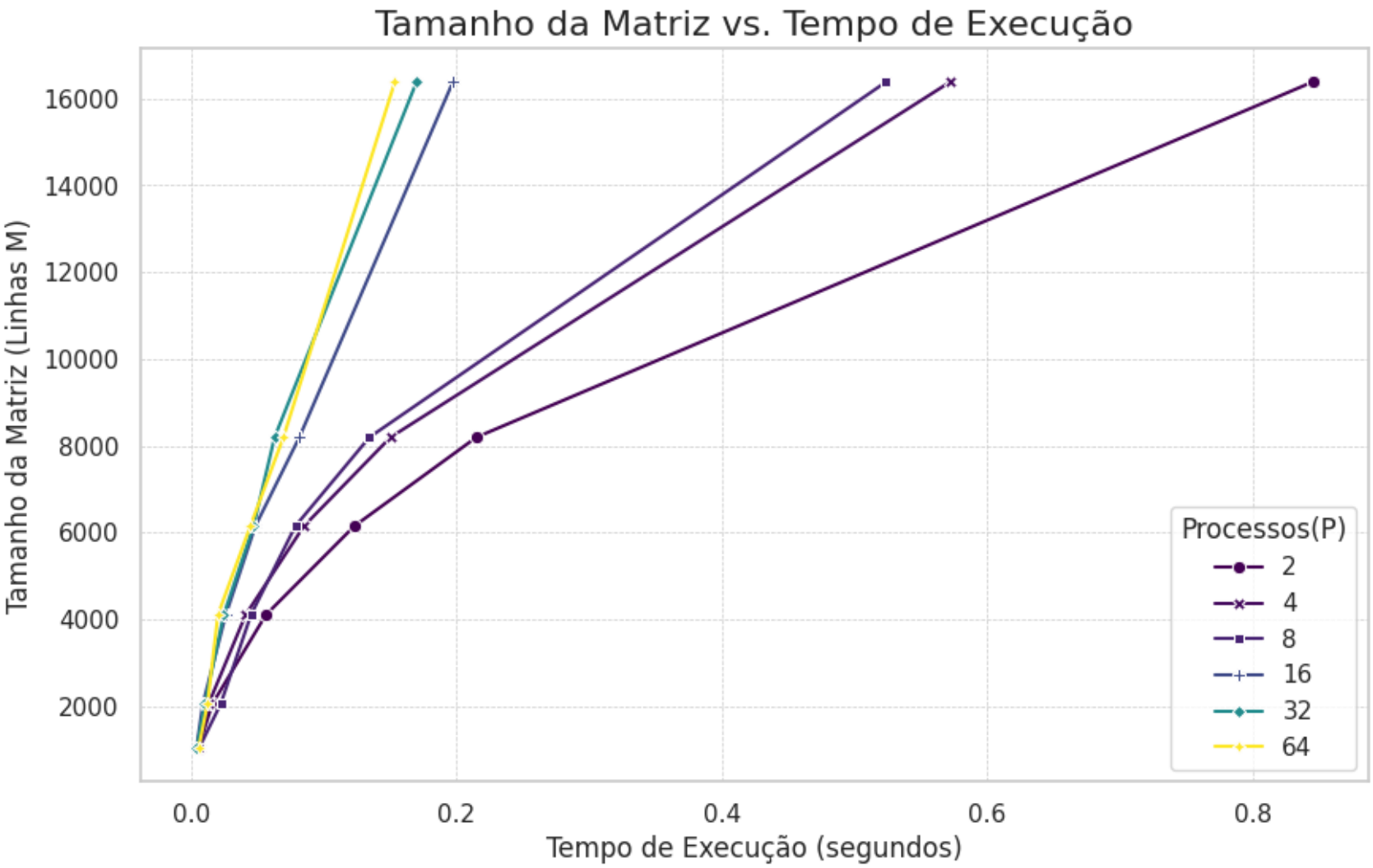

Figura: Comparativo do tempo de execução para cada tamanho de matriz

Análise dos Resultados

- **Tempo de Execução:** Conforme esperado, o tempo de execução diminuiu consistentemente com o aumento do número de processos para **todos os tamanhos de matriz**. O maior ganho de velocidade foi observado ao passar de 2 para 16 processos.
- **Eficiência e Granularidade:** A eficiência da paralelização foi mais baixa para a matriz menor (1024x1024). Isso ocorre porque a *granularidade* do problema é baixa: o tempo de comunicação da MPI é significativo em comparação com o pouco trabalho de cálculo que cada processo realiza.
- **Ganhos Decrescentes:** Para as matrizes maiores (a partir de 8192x8192), o ganho de desempenho ao saltar de 32 para 64 processos foi menor do que os saltos iniciais. Isso indica que, com muitos processos, o *overhead* de comunicação das cláusulas `MPI_Scatter` e `MPI_Gather` começa a limitar a escalabilidade.

Conclusão

- A paralelização do produto matriz-vetor com MPI foi implementada com sucesso, demonstrando uma redução efetiva no tempo de execução em todos os cenários testados.
- Os resultados confirmam um princípio fundamental da computação paralela: a escalabilidade é limitada pelo *overhead* de comunicação. Mesmo sem observar um *slowdown*, a eficiência diminui à medida que a comunicação se torna um gargalo.
- Para o maior problema testado (matriz 16384x16384), o speedup alcançou um fator superior a **5.5x** (comparado à base de 2 processos), provando que a abordagem distribuída é altamente vantajosa para problemas de grande escala.

matrix_vetor.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include <time.h>

// Protótipo para inicialização
void initialize_matrix_and_vector(double *A, double *x, int M, int N);

int main(int argc, char *argv[]) {
    int rank, size;
    double start_time, end_time;

    // Inicialização do MPI
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // --- AJUSTE 1: Receber apenas um argumento (M) ---
    if (argc < 2) {
        if (rank == 0) {
            fprintf(stderr, "Uso: %s <M_tamanho_matriz_quadrada>\n", argv[0]);
        }
        MPI_Finalize();
        return 1;
    }

    int M_GLOBAL = atoi(argv[1]);
    int N_GLOBAL = M_GLOBAL; // Força a matriz a ser quadrada
    // ----------------------------------------------------

    // Validação da divisibilidade
    if (rank == 0) {
        if (M_GLOBAL % size != 0) {
            fprintf(stderr, "ERRO: O tamanho da matriz M (%d) não é divisível pelo número de processos (%d).\n", M_GLOBAL, size);
            fprintf(stderr, "Por favor, ajuste os parâmetros para garantir a divisibilidade.\n");
            MPI_Abort(MPI_COMM_WORLD, 1);
        }
    }

    int local_M = M_GLOBAL / size;

    // --- Alocação de Memória ---
    double *A_global = NULL;
    double *x_global = NULL;
    double *y_global = NULL;

    if (rank == 0) {
        A_global = (double *)malloc(M_GLOBAL * N_GLOBAL * sizeof(double));
        x_global = (double *)malloc(N_GLOBAL * sizeof(double));
```

```c
        y_global = (double *)malloc(M_GLOBAL * sizeof(double));

        initialize_matrix_and_vector(A_global, x_global, M_GLOBAL, N_GLOBAL);

        start_time = MPI_Wtime();
    }

    // Alocação em Todos os Processos
    double *A_local = (double *)malloc(local_M * N_GLOBAL * sizeof(double));
    x_global = (double *)malloc(N_GLOBAL * sizeof(double));
    double *y_local = (double *)malloc(local_M * sizeof(double));

    // 1. DISTRIBUIÇÃO DO VETOR x com MPI_Bcast
    MPI_Bcast(x_global, N_GLOBAL, MPI_DOUBLE, 0, MPI_COMM_WORLD);

    // 2. DISTRIBUIÇÃO DA MATRIZ A com MPI_Scatter
    MPI_Scatter(
        A_global,
        local_M * N_GLOBAL,
        MPI_DOUBLE,
        A_local,
        local_M * N_GLOBAL,
        MPI_DOUBLE,
        0,
        MPI_COMM_WORLD
    );

    // 3. CÁLCULO LOCAL
    for (int i = 0; i < local_M; i++) {
        y_local[i] = 0.0;
        for (int j = 0; j < N_GLOBAL; j++) {
            y_local[i] += A_local[i * N_GLOBAL + j] * x_global[j];
        }
    }

    // 4. AGREGAÇÃO DO RESULTADO com MPI_Gather
    MPI_Gather(
        y_local,
        local_M,
        MPI_DOUBLE,
        y_global,
        local_M,
        MPI_DOUBLE,
        0,
        MPI_COMM_WORLD
    );

    // 5. FINALIZAÇÃO e Análise de Tempo
    if (rank == 0) {
        end_time = MPI_Wtime();
        double elapsed_time = end_time - start_time;

        // --- AJUSTE 2: Atualizar a mensagem de saída ---
```

```c
        printf("--- Produto Matriz-Vetor Distribuído (Matriz Quadrada) ---\n");
        printf("Tamanho da Matriz A: %d x %d\n", M_GLOBAL, M_GLOBAL);
        printf("Número de Processos: %d\n", size);
        printf("Linhas por Processo (M/size): %d\n", local_M);
        printf("Tempo de Execução (Computação + Comunicação): %.6f segundos\n", elapsed_time);
        // ---------------------------------------------
    }

    // --- Liberação de Memória ---
    free(A_local);
    free(y_local);
    free(x_global);
    if (rank == 0) {
        free(A_global);
        free(y_global);
    }

    MPI_Finalize();
    return 0;
}

// Função auxiliar para inicializar a matriz e o vetor
void initialize_matrix_and_vector(double *A, double *x, int M, int N) {
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            A[i * N + j] = (double)(i % 10 + j % 10) / 10.0;
        }
    }
    for (int j = 0; j < N; j++) {
        x[j] = 1.0;
    }
}

```

Benchmark.sh

```bash
#!/bin/bash

#SBATCH --partition=amd-512  # partição para a qual o job é enviado
#SBATCH --exclusive
#SBATCH --time=0-0:20
#SBATCH --nodes=2 #número de nós
#SBATCH --ntasks-per-node=64 #número de tarefas por nós

# ========================================================================
# CONFIGURAÇÕES
# ========================================================================

# Nome do seu executável MPI (ex: ./matriz_vetor_quadrada)
MPI_EXEC="./matrix_vetor"

# Arquivo de saída CSV para registrar os resultados
OUTPUT_FILE="benchmark_results_final.csv"

# Array com os NÚMEROS DE PROCESSOS (P) a serem testados
NUM_PROCS_ARRAY=(2 4 8 16 32 64)

# AJUSTE 1: Lista de tamanhos de matriz (M) onde todos os valores
# são múltiplos de 64 para garantir a divisibilidade.
MATRIX_SIZES=(1024 2048 4096 6144 8192 16384)


# ========================================================================
# FUNÇÃO DE CAPTURA DO TEMPO
# ========================================================================
function run_benchmark {
    local M=$1
    local PROCS=$2

    TIME_OUTPUT=$(mpirun -np "$PROCS" "$MPI_EXEC" "$M" 2>&1 | \
                  grep "Tempo de Execução" | \
                  sed 's/.*: //; s/ segundos//')

    if [[ "$TIME_OUTPUT" =~ ^[0-9].*\.[0-9]*$ ]]; then
        echo "${M},${M},${PROCS},${TIME_OUTPUT}" >> "$OUTPUT_FILE"
        echo "  [OK] P=$PROCS | Tempo: ${TIME_OUTPUT}s"
    else
        echo "  [FAIL] P=$PROCS | Erro na captura do tempo. Saída: '$TIME_OUTPUT'"
    fi
}


# ========================================================================
# FLUXO PRINCIPAL
# ========================================================================
```

```

echo "Iniciando o Benchmark MPI. Resultados em $OUTPUT_FILE"

# Verifica se o executável existe
if [ ! -f "$MPI_EXEC" ]; then
    echo "ERRO: Executável '$MPI_EXEC' não encontrado. Compile o código C primeiro."
    exit 1
fi

# Cria/limpa o arquivo de saída e adiciona o cabeçalho CSV
echo "Tamanho(M),Tamanho(N),Processos(P),Tempo(s)" > "$OUTPUT_FILE"

# Loops aninhados para testar todas as combinações
for M in "${MATRIX_SIZES[@]}"; do
    echo -e "\n--- Testando Matriz ${M}x${M} ---"

    for PROCS in "${NUM_PROCS_ARRAY[@]}"; do
        # AJUSTE 2: A verificação de divisibilidade foi removida,
        # pois garantimos que todas as combinações são válidas.

        run_benchmark "$M" "$PROCS"
    done
done

echo -e "\nBenchmark concluído."
echo "Para analisar os resultados, use o arquivo '$OUTPUT_FILE' com seu script Python."

```